

# Luma Programming Language

Language Specification and Reference Manual

Version 1.0

Developed by Debashis Baral

2025

# Table of Contents

---

|                                       |       |
|---------------------------------------|-------|
| <b>1. Introduction</b>                | 3     |
| 1.1 Overview                          | 3     |
| 1.2 Design Philosophy                 | 3     |
| <b>2. Language Fundamentals</b>       | 3     |
| 2.1 Program Structure                 | 3     |
| 2.2 Lexical Elements                  | 4     |
| 2.3 Data Types                        | 4     |
| <b>3. Variables and Scope</b>         | 5     |
| 3.1 Variable Declaration              | 5     |
| 3.2 Variable Scope                    | 5     |
| <b>4. Statements and Expressions</b>  | 6     |
| 4.1 Control Flow                      | 6     |
| 4.2 Operators                         | 6     |
| <b>5. Subroutines</b>                 | 7     |
| 5.1 Functions                         | 7     |
| 5.2 Methods                           | 7     |
| <b>6. Object-Oriented Programming</b> | 8     |
| 6.1 Classes                           | 8     |
| 6.2 Constructors                      | 8     |
| <b>7. Standard Library</b>            | 9     |
| <b>8. Example Programs</b>            | 10-11 |

# 1. Introduction

---

## 1.1 Overview

Luma is a high-level, object-oriented programming language designed for educational purposes and system-level programming. This document provides a complete specification of the Luma programming language, including syntax, semantics, and standard library functions. It is intended for programmers who wish to write Luma programs and compiler developers implementing Luma language tools.

## 1.2 Design Philosophy

Luma is designed with the following principles in mind:

- **Simplicity:** The language features a minimal set of constructs that are easy to learn and understand.
- **Type Safety:** Luma is statically typed, with three primitive types and user-defined classes.
- **Object Orientation:** All functionality is organized into classes, promoting modular and reusable code.
- **Efficiency:** The language is designed to compile to efficient virtual machine code suitable for embedded systems.

# 2. Language Fundamentals

---

## 2.1 Program Structure

A Luma program consists of one or more class declarations. Each class resides in a separate source file with the `.luma` extension. The filename must match the class name. Every Luma program must contain a class named `Main` with a function named `main`, which serves as the program entry point.

Basic program structure:

```
class ClassName {
    field type fieldName;
    constructor ClassName methodName(parameters) {
        // constructor body
    }
    function type functionName(parameters) {
        // function body
    }
    method type methodName(parameters) {
        // method body
    }
}
```

## 2.2 Lexical Elements

### Keywords

The following are reserved keywords in Luma:

|       |             |          |        |       |
|-------|-------------|----------|--------|-------|
| class | constructor | function | method | field |
| var   | static      | let      | do     | if    |
| else  | while       | return   | true   | false |
| null  | this        |          |        |       |

### Identifiers

Identifiers are sequences of letters, digits, and underscores that must begin with a letter. Identifiers are case-sensitive. Examples: `counter`, `myVariable`, `x1`.

### Comments

Luma supports single-line and multi-line comments:

```
// This is a single-line comment

/** This is a multi-line comment
    spanning multiple lines */
```

## 2.3 Data Types

Luma supports three primitive data types and user-defined class types.

### Primitive Types

| Type                 | Description           | Value Range                             |
|----------------------|-----------------------|-----------------------------------------|
| <code>int</code>     | 16-bit signed integer | -32768 to 32767                         |
| <code>char</code>    | Unicode character     | 0 to 255                                |
| <code>boolean</code> | Boolean value         | <code>true</code> or <code>false</code> |

### Class Types

Every class declaration defines a new object type. Variables of class type hold references to objects (similar to pointers). The special value `null` represents a null reference.

### Arrays

Luma supports one-dimensional arrays of any type. Arrays are declared using the syntax `Array` and are dynamically allocated. Array indexing is zero-based.

```
var Array myArray;
let myArray = Array.new(10); // Create array of size 10
let myArray[0] = 42;        // Set first element
```

### 3. Variables and Scope

#### 3.1 Variable Declaration

Luma supports three kinds of variables, each with different scope and lifetime characteristics.

##### Field Variables

Field variables are declared at the class level using the `field` keyword. Each object instance has its own copy of field variables.

```
class Point {
    field int x, y;
    // ...
}
```

##### Static Variables

Static variables are shared among all instances of a class. They are declared using the `static` keyword.

```
class MathUtils {
    static int lastResult;
    // ...
}
```

##### Local Variables

Local variables are declared within subroutines using the `var` keyword. They exist only during the execution of the subroutine.

```
function int compute(int a, int b) {
    var int result;
    let result = a + b;
    return result;
}
```

#### 3.2 Variable Scope

The scope of a variable determines where it can be accessed:

- **Field variables:** Accessible within all methods of the class using the `this` keyword.
- **Static variables:** Accessible within all subroutines of the class using the class name.
- **Local variables:** Accessible only within the subroutine where they are declared.
- **Parameter variables:** Accessible within the subroutine as local variables.

| Variable Kind | Declaration               | Lifetime             | Scope            |
|---------------|---------------------------|----------------------|------------------|
| Static        | <code>static</code>       | Program duration     | Class-level      |
| Field         | <code>field</code>        | Object lifetime      | Instance methods |
| Local         | <code>var</code>          | Subroutine execution | Subroutine body  |
| Parameter     | Function/method signature | Subroutine execution | Subroutine body  |

# 4. Statements and Expressions

## 4.1 Control Flow

### Let Statement

The `let` statement assigns a value to a variable or array element.

```
let x = 10;
let arr[i] = x + 5;
```

### If Statement

Conditional execution based on a boolean expression.

```
if (x > 0) {
    // positive case
} else {
    // negative or zero case
}
```

### While Statement

Repetitive execution while a condition is true.

```
while (i < 10) {
    let sum = sum + i;
    let i = i + 1;
}
```

### Do Statement

Calls a subroutine and discards the return value.

```
do Output.println("Hello World");
do Memory.deAlloc(this);
```

### Return Statement

Returns control from a subroutine, optionally with a value.

```
return x + y;
return;
```

## 4.2 Operators

Luma supports standard arithmetic, logical, and comparison operators.

| Category   | Operators                                                   | Description                                     |
|------------|-------------------------------------------------------------|-------------------------------------------------|
| Arithmetic | <code>+</code> <code>-</code> <code>*</code> <code>/</code> | Addition, subtraction, multiplication, division |
| Comparison | <code>=</code> <code>&lt;</code> <code>&gt;</code>          | Equal, less than, greater than                  |
| Logical    | <code>&amp;</code> <code> </code> <code>~</code>            | AND, OR, NOT                                    |
| Unary      | <code>-</code> <code>~</code>                               | Negation, bitwise NOT                           |

## 5. Subroutines

---

### 5.1 Functions

Functions are class-level subroutines that do not operate on object instances. They are declared using the **function** keyword and are called using the class name.

```
class Math {  
    function int multiply(int a, int b) {  
        return a * b;  
    }  
}  
  
// Usage:  
  
let result = Math.multiply(5, 3);
```

### 5.2 Methods

Methods operate on object instances and have access to the object's fields through the implicit **this** reference. They are declared using the **method** keyword.

```
class Point {  
    field int x, y;  
  
    method int getX() {  
        return x;  
    }  
  
    method void setX(int newX) {  
        let x = newX;  
        return;  
    }  
}
```

## 6. Object-Oriented Programming

---

### 6.1 Classes

A class encapsulates data (fields) and behavior (methods). Classes serve as templates for creating objects.

### 6.2 Constructors

Constructors are special methods that initialize new objects. They are declared using the **constructor** keyword and must return **this**.

```
class Fraction {
    field int numerator, denominator;

    constructor Fraction new(int num, int den) {
        let numerator = num;
        let denominator = den;
        return this;
    }
}

// Usage:
var Fraction f;
let f = Fraction.new(3, 4);
```

### Object Disposal

Objects should be explicitly deallocated using **Memory.deAlloc()** when no longer needed.

```
method void dispose() {
    do Memory.deAlloc(this);
    return;
}
```



## 7. Standard Library

---

Luma includes a standard library providing essential functionality. All programs have access to these built-in classes.

### Math Class

Provides mathematical operations:

- `function int multiply(int x, int y)` - Returns  $x \times y$
- `function int divide(int x, int y)` - Returns  $x \div y$  (integer division)
- `function int min(int x, int y)` - Returns the minimum value
- `function int max(int x, int y)` - Returns the maximum value
- `function int sqrt(int x)` - Returns the integer square root of  $x$
- `function int abs(int x)` - Returns the absolute value of  $x$

### String Class

Implements string operations:

- `constructor String new(int maxLength)` - Creates a new string
- `method int length()` - Returns the length of the string
- `method char charAt(int j)` - Returns the character at position  $j$
- `method void setCharAt(int j, char c)` - Sets the character at position  $j$
- `method String appendChar(char c)` - Appends a character to the string
- `method void eraseLastChar()` - Removes the last character
- `method int intValue()` - Returns the integer value of the string
- `method void setInt(int n)` - Sets the string to represent integer  $n$

### Array Class

Provides array construction and disposal:

- `function Array new(int size)` - Creates a new array of the given size
- `method void dispose()` - Disposes the array

### Output Class

Handles text output to the screen:

- `function void moveCursor(int i, int j)` - Moves cursor to row  $i$ , column  $j$
- `function void printChar(char c)` - Prints a character at the cursor
- `function void printString(String s)` - Prints a string at the cursor
- `function void printInt(int i)` - Prints an integer at the cursor
- `function void println()` - Advances to the next line
- `function void backSpace()` - Moves cursor back one position

### Keyboard Class

Handles keyboard input:

- `function char keyPressed()` - Returns the currently pressed key
- `function char readChar()` - Reads and returns the next character
- `function String readLine(String message)` - Reads a line of text
- `function int readInt(String message)` - Reads and returns an integer

### Memory Class

Provides direct memory access and allocation:

- `function int peek(int address)` - Returns the value at memory address
- `function void poke(int address, int value)` - Sets memory at address
- `function Array alloc(int size)` - Allocates memory block of given size
- `function void deAlloc(Array object)` - Frees memory block

### Sys Class

Provides system-level services:

- `function void halt()` - Halts program execution
- `function void error(int errorCode)` - Prints error code and halts
- `function void wait(int duration)` - Waits for specified duration

## 8. Example Programs

---

### Example 1: Hello World

The classic first program demonstrates basic output:

```
/** Simple Hello World program */
class Main {
    function void main() {
        do Output.printString("Hello World!");
        do Output.println();
        return;
    }
}
```

### Example 2: Array Processing

This example demonstrates array creation, manipulation, and iteration:

```
/** Computes the average of array elements */
class Main {
    function void main() {
        var Array numbers;
        var int i, sum, avg;

        let numbers = Array.new(5);
        let i = 0;

        // Initialize array
        while (i < 5) {
            let numbers[i] = (i + 1) * 10;
            let i = i + 1;
        }

        // Calculate sum
        let sum = 0;
        let i = 0;
        while (i < 5) {
            let sum = sum + numbers[i];
            let i = i + 1;
        }

        let avg = sum / 5;
        do Output.printString("Average: ");
        do Output.printInt(avg);
        do Output.println();

        do numbers.dispose();
        return;
    }
}
```

### Example 3: Object-Oriented Programming

This example demonstrates a complete class with constructor, methods, and proper memory management:

```
/** Represents a point in 2D space */
class Point {
    field int x, y;
    /** Creates a new point at (ax, ay) */
    constructor Point new(int ax, int ay) {
        let x = ax;
        let y = ay;
        return this;
    }
    /** Disposes this point */
    method void dispose() {
        do Memory.deAlloc(this);
        return;
    }
    /** Returns the x coordinate */
    method int getX() {
        return x;
    }
    /** Returns the y coordinate */
    method int getY() {
        return y;
    }
    /** Moves the point by (dx, dy) */
    method void move(int dx, int dy) {
        let x = x + dx;
        let y = y + dy;
        return;
    }
    /** Prints the point coordinates */
    method void print() {
        do Output.printString("(");
        do Output.printInt(x);
        do Output.printString(", ");
        do Output.printInt(y);
        do Output.printString(")");
        return;
    }
}

/** Test program for Point class */
class Main {
    function void main() {
        var Point p;

        let p = Point.new(10, 20);
        do p.print();
        do Output.println();

        do p.move(5, -3);
        do p.print();
        do Output.println();

        do p.dispose();
        return;
    }
}
```